IoT Based Server and IT Infrastructure Health Monitoring

Software Only IoT Mini Project Report

Team 11

B.Tech Computer Science and Engineering

Semester 7

Team members and roll numbers: add before submission

Contents

Abstract
1 Problem definition
2 Objectives
3 System architecture
4 Dataset design
5 Exploratory workload analysis
6 Model development
7 Advanced operational analytics
8 Decision and control policy
9 Software implementation
10 Evaluation results
11 Verification
12 Two-laptop demonstration
13 Limitations
14 Future work
15 Conclusion
References

Abstract

This project implements a software-only IoT system that monitors simulated servers, classifies their operating condition, forecasts resource demand and issues safe resource-management commands. A Python replay service publishes one-minute telemetry through MQTT. Node-RED validates and routes each message. The analytics node compares traditional machine learning, ensemble learning and a neural baseline, then uses the selected Random Forest model for live inference. Ridge regression predicts CPU and memory usage 15 minutes ahead. An auditable policy converts predictions and safety thresholds into commands for a software digital-twin actuator. SQLite preserves telemetry, predictions, explanations and commands, while a light-theme dashboard presents the current fleet state and evaluation evidence.

The fixed dataset contains 80,640 readings from four simulated servers over 14 days. Chronological splitting assigns the first 10 days to training, the next two days to validation and the final two days to testing. Random Forest achieved 99.33% test accuracy and 0.947 macro-F1 after selection by validation macro-F1. CPU and memory forecasts achieved mean absolute errors of 4.48 and 1.93 percentage points. The project also implements a deterministic health score, forecast capacity headroom, local prediction explanations, rolling drift detection and telemetry completeness monitoring.

1 Problem definition

Server workloads change over time and may cause overload, wasted capacity, thermal stress or service failure. Operators need more than a threshold alarm. They need a continuous view of resource usage, an estimate of future demand and an auditable recommendation that explains why an action is appropriate.

The project addresses this need with a complete IoT feedback loop. Software sensors publish CPU, memory, disk, network, temperature and service measurements. The analytics service identifies normal, underutilized, overloaded and anomalous conditions. The decision engine can recommend workload redistribution, standby capacity, service restart, cooling, disk cleanup, rate limiting or consolidation. No physical hardware or production server is modified.

2 Objectives

1. Build a reproducible server telemetry dataset with normal workloads and identifiable fault scenarios.

2. Compare traditional, ensemble and neural classification approaches without leaking labels or future targets into model features.

3. Forecast CPU and memory requirements 15 minutes ahead.

4. Implement MQTT communication and a real Node-RED processing dependency.

5. Store telemetry, predictions and decisions in an auditable database.

6. Complete the feedback loop with a safe digital-twin actuator.

7. Add operational features for health scoring, explainability, capacity headroom, drift detection and data-quality monitoring.

8. Support a repeatable two-laptop demonstration with one command per computer.

3 System architecture

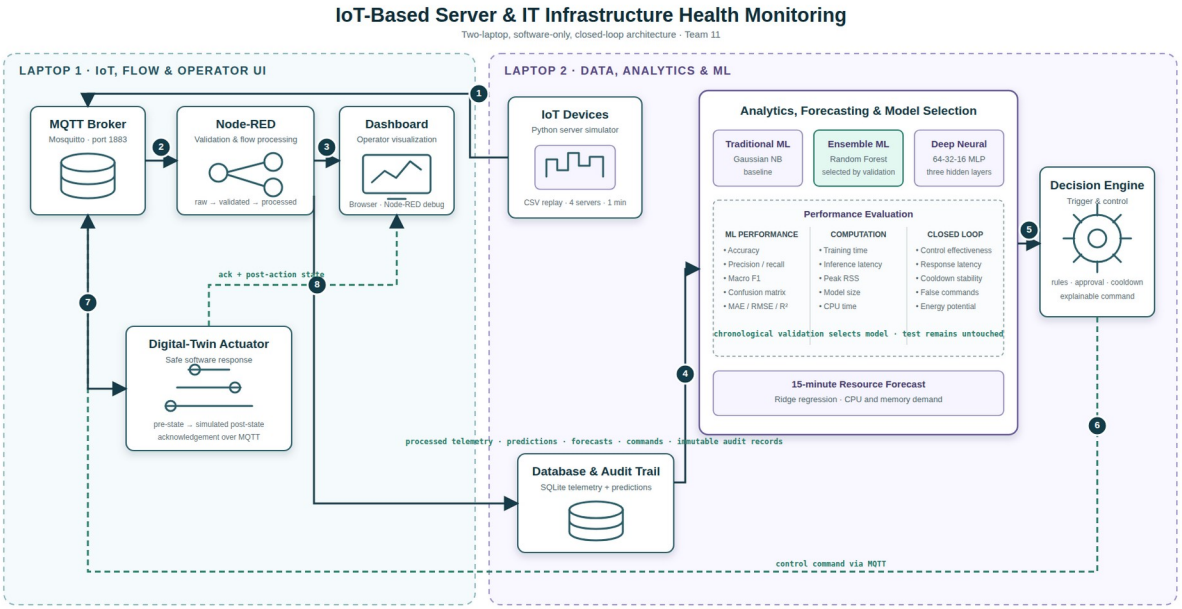


Figure 1 Two-laptop closed-loop IoT architecture

The solution separates IoT transport from analytics. The other PC runs Mosquitto, Node-RED and the software actuator. The main laptop holds the fixed dataset, model-training pipeline, live publisher, analytics subscriber, SQLite database and dashboard server. Both computers connect through the same local network.

The end-to-end message path is:

1. The Python simulator reads a fixed CSV record and publishes raw telemetry to `team11/servers/{server_id}/telemetry`.

2. Node-RED validates required fields and republishes accepted messages to `team11/servers/{server_id}/processed`.

3. The analytics subscriber classifies condition, forecasts demand, calculates advanced operational indicators and writes one audit record.

4. Predictions publish to `team11/servers/{server_id}/prediction`.

5. The policy engine publishes necessary commands to `team11/servers/{server_id}/command`.

6. The digital-twin actuator returns a simulated acknowledgement and post-action state without modifying any physical server.

Node-RED is a functional dependency. Stopping its flow stops processed telemetry from reaching the ML service.

4 Dataset design

4.1 Scope and sampling

The primary dataset is `server_health_synthetic.csv`. It is synthetic, scenario-based and reproducible. It must not be described as production or real-world infrastructure data.

Property	Value
Simulated servers	4
Duration	14 days
Sampling interval	60 seconds
Total readings	80,640
Training readings	57,600
Validation readings	11,520
Test readings	11,520

4.2 Telemetry fields

Each record contains timestamp, server identifier, CPU usage, memory usage, disk usage, inbound and outbound network rate, temperature, estimated power, one-minute load, response time, error rate, service heartbeat and active sessions. Ground-truth fields identify the scenario, fault, condition and future targets for offline evaluation only.

4.3 Condition and fault coverage

The dataset contains 56,213 normal rows, 22,103 underutilized rows, 1,202 overloaded rows and 1,122 anomalous rows. Six injected fault families cover CPU stress, memory leak, disk saturation, network storm, cooling fault and service failure. The remaining 79,518 records have no injected fault.

4.4 Leakage prevention

The classifier uses current sensor values, cyclic hour features and weekday. The following columns never enter the feature vector: condition label, fault type, recommended action, scenario identifier, future CPU and future memory. Automated tests enforce this boundary.

5 Exploratory workload analysis

CPU usage has a 0.975 correlation with temperature and a 0.993 correlation with estimated power. Active sessions and inbound network traffic have a 0.618 correlation. The highest average CPU hour is 11:00 UTC, while the quietest is 01:00 UTC. These patterns support time-aware model features and demonstrate that the simulator preserves realistic relationships among workload, thermal and energy variables.

6 Model development

6.1 Classification models

The project compares three model families under the same chronological split.

Model	Family	Validation macro-F1	Test accuracy	Test macro-F1
Gaussian Naive Bayes	Traditional ML	0.810	89.79%	0.838
Random Forest	Ensemble ML	0.994	99.33%	0.947
MLP 64 32 16	Neural baseline	0.846	96.46%	0.868

Random Forest won selection because it produced the highest validation macro-F1. The test set did not influence model selection. Macro-F1 is the main selection metric because accuracy alone can hide weak results on rare overload and anomaly classes.

6.2 Selected model class performance

Condition	Precision	Recall	F1	Test support
Underutilized	0.999	0.999	0.999	2,967
Normal	0.997	1.000	0.999	7,927
Overloaded	0.815	0.959	0.881	244
Anomalous	1.000	0.830	0.907	382

The selected model performs strongly overall. Anomalous recall remains lower than the other classes because some anomalous rows resemble overloaded conditions. Deterministic safety rules supplement the statistical classifier for service, thermal, disk, network and error emergencies.

6.3 Fifteen-minute forecasting

Ridge regression predicts CPU and memory utilization 15 minutes ahead from the same current telemetry and time features.

Target	MAE	RMSE	R squared	Test rows
CPU percentage	4.48	5.80	0.870	11,460
Memory percentage	1.93	2.88	0.939	11,460

The dashboard draws measured values as solid lines and forecasts as dashed lines. Forecast headroom equals 100 minus the maximum of predicted CPU, predicted memory and current disk utilization.

7 Advanced operational analytics

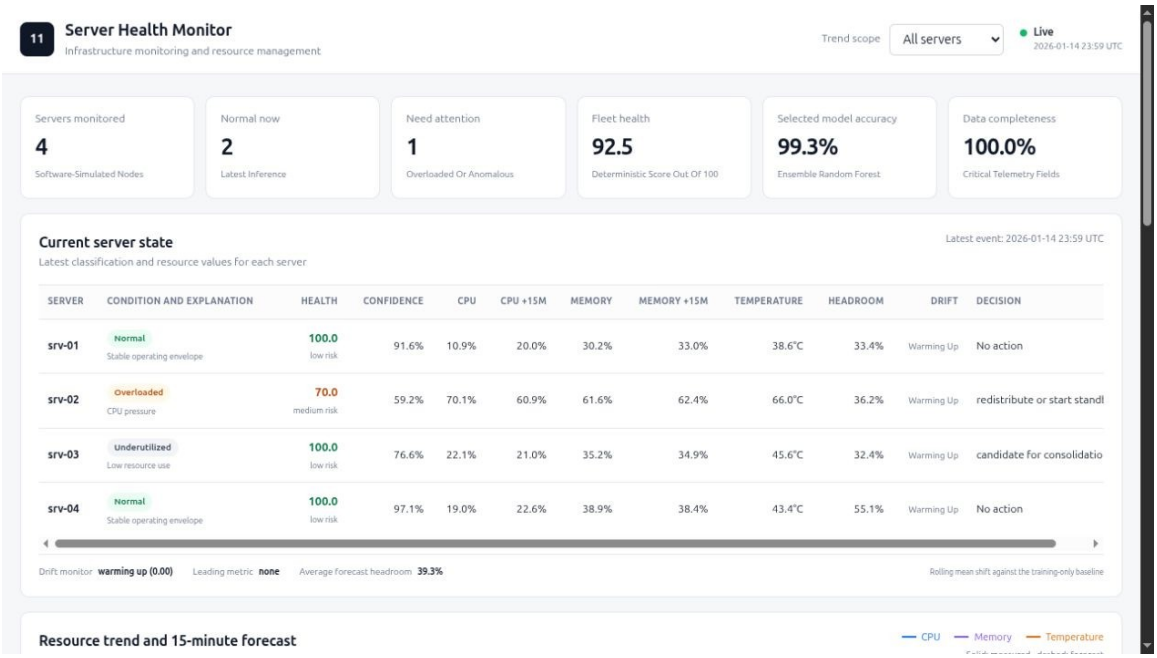


Figure 2 Enhanced operational dashboard with mixed fleet state

7.1 Health score

The deterministic health score ranges from 0 to 100. Penalties represent CPU, memory, disk, thermal, response-time, error-rate and service-heartbeat pressure. The predicted condition limits the maximum score for overloaded and anomalous states. This score is an auditable operational summary, not another trained classifier.

7.2 Local explanation

Every prediction stores up to three strongest pressure factors with observed values and contributions. Underutilized servers receive a low-resource-use explanation. Stable servers show that no pressure threshold is active. The dashboard displays the primary factor beside the predicted condition.

7.3 Concept drift detection

The analysis pipeline calculates means and standard deviations from the training split only. During live operation, a 120-message rolling window compares recent metric means with this baseline. The maximum standardized mean shift becomes the drift score. Scores below 1.5 are stable, scores from 1.5 to 2.5 require observation and scores of at least 2.5 indicate drift. The monitor reports a warm-up state until 20 messages arrive.

7.4 Data quality

The advanced API calculates completeness across CPU, memory, disk, temperature, response time, predicted condition and confidence for the latest server rows. Node-RED rejects malformed telemetry before inference, and the dashboard makes the remaining completeness visible.

8 Decision and control policy

Safety-critical sensor rules take priority over model predictions.

Trigger	Decision	Approval
Service heartbeat down	Restart service	Automatic simulation
Temperature at least 82 C	Increase cooling and alert	Automatic simulation
Disk usage at least 92%	Clear disk and alert	Operator required
Inbound traffic at least 600 Mbit/s	Rate limit and inspect	Automatic simulation
Error rate at least 12%	Isolate process and redistribute	Automatic simulation
Anomalous prediction	Inspect and redistribute	Automatic simulation
Overload or high CPU or memory	Redistribute or start standby	Automatic simulation
Low CPU and memory	Consolidation candidate	Operator required

Every command contains a UUID, timestamp, server identifier, reason, severity, observed state, expected effect and approval flag. A five-minute per-server and per-action cooldown suppresses repeated commands.

9 Software implementation

Python implements simulation, feature conversion, models, policy, replay, storage, API and test automation. Mosquitto carries MQTT messages. Node-RED provides flow validation and visible topic routing. SQLite stores immutable telemetry and command evidence. The dashboard uses dependency-free HTML, CSS, JavaScript and Canvas rendering.

The repository includes a single launcher with three modes. `edge` starts the other PC, `main` trains and demonstrates from the presentation laptop, and `local` provides a one-computer backup. The launcher supports saved-model reuse, custom dashboard ports, replay length and replay speed through environment variables.

10 Evaluation results

The closed-loop evaluator processed 11,520 untouched test messages and issued 746 commands. It detected all six fault families, suppressed 2,296 repeated commands through cooldown and issued no unnecessary command on rows labelled normal. Mean policy decision latency was approximately 0.017 milliseconds. The digital twin reported 100% simulated control effectiveness for supported actions. The estimated energy-saving potential was 2.25 kWh under the documented assumption that consolidation avoids 35% of per-minute power during underutilized intervals.

These control and energy results describe a simulation. They do not demonstrate production savings or physical recovery.

11 Verification

Nine automated tests currently verify feature leakage prevention, prediction contracts, advanced indicator ranges, drift detection, policy priority, SQLite persistence, command cooldown, saved performance thresholds, validation-only model selection and closed-loop completeness. The one-command launcher also completed an end-to-end MQTT test through Node-RED, the analytics subscriber, SQLite, dashboard and digital-twin actuator.

12 Two-laptop demonstration

The other PC starts first with `./team11.sh edge`. It prints the Wi-Fi address used by the main command. The presentation laptop then runs `./team11.sh main OTHER_PC_IP`. The main command creates dependencies when needed, validates the dataset, rebuilds analytics and models, runs tests, starts the live services and publishes a balanced demonstration stream. A final mixed state leaves two normal servers, one overloaded server and one underutilized server visible for discussion.

The dashboard is available at `http://127.0.0.1:8080` on the main laptop. Node-RED is available at `http://OTHER_PC_IP:1880`. If the network or second computer fails, `./team11.sh local` starts the complete architecture on one computer.

13 Limitations

The dataset is synthetic and cannot establish external validity on production infrastructure. The digital twin estimates post-action state through defined rules rather than observed physical outcomes. MQTT uses anonymous classroom configuration without TLS. The classifier works on individual readings rather than a temporal neural sequence. Drift detection measures standardized rolling mean shift and does not cover every form of distribution change.

14 Future work

Future validation should replay public cluster traces or collect consented telemetry from a controlled laboratory server. A production deployment should use authenticated MQTT over TLS, role-based dashboard access and signed command messages. Sequence models can be compared when sufficient real temporal data is available. Control effectiveness should be measured through container or virtual-machine orchestration in an isolated test environment.

15 Conclusion

The project implements the full IoT cycle from software sensing through communication, processing, analytics, decision and simulated action. It compares three classifier families, preserves a chronological test boundary, forecasts resource demand and exposes auditable control decisions. The advanced features add operational value without hiding their assumptions. The resulting system is suitable for a live software-only evaluation and provides clear paths for later real-infrastructure validation.

References

1. Project dataset and source code in the Team 11 submission package.
2. OASIS MQTT Version 5.0 specification.
3. Node-RED documentation.
4. Eclipse Mosquitto documentation.
5. Scikit-learn documentation for Random Forest, MLP and Ridge regression.